

Revisión técnica del TFM

*Arquitectura inteligente de datos y modelos predictivos
para la producción de leche «a la carta»*

Secciones revisadas: Materiales y métodos · Resultados

Basado en evidencia directa del código fuente del proyecto Tools4Milk MVP

3. Materiales y métodos

3.1. Enfoque metodológico

El desarrollo del sistema Tools4Milk se abordó siguiendo una metodología iterativa e incremental, orientada a la construcción de un producto mínimo viable (MVP) funcional que permitiese validar la viabilidad técnica de una plataforma de gestión integral para explotaciones lecheras. Se priorizó la implementación de módulos operativos reales — gestión de animales, tareas, alertas, turnos, incidencias, calidad de leche y predicciones heurísticas— frente a la integración de modelos de aprendizaje automático, cuyo diseño queda reservado para fases futuras del proyecto.

El dominio de aplicación se centra en una explotación ganadera de vacuno lechero ubicada en Villalba (Lugo), con aproximadamente 300 cabezas en ordeño y un sistema de ordeño robotizado. La elección de esta explotación como caso de estudio responde a la disponibilidad de datos operativos reales y a la colaboración directa del personal técnico de la granja durante el diseño de requisitos.

3.2. Arquitectura del sistema

3.2.1. Arquitectura general

El sistema adopta una arquitectura monolítica modular desplegada mediante contenedores Docker. A diferencia de aproximaciones basadas en microservicios, se optó por una única aplicación backend que concentra toda la lógica de negocio, simplificando el despliegue y el mantenimiento en el contexto de un MVP. La comunicación entre frontend y backend se realiza exclusivamente a través de una API REST protegida por tokens JWT.

La infraestructura de despliegue se define en un archivo `docker-compose.yml` que orquesta cuatro servicios:

Servicio	Imagen / Framework	Función
db	postgres:15-alpine	Base de datos relacional PostgreSQL 15

backend	FastAPI + Uvicorn	API REST monolítica (lógica de negocio)
frontend	Next.js 16 + React 19	Interfaz de usuario SPA con SSR
nginx	nginx:alpine	Proxy inverso y punto de entrada HTTP

El proxy inverso Nginx enruta las peticiones `/api/*` al backend (puerto 8000) y el resto al frontend (puerto 3000), proporcionando un punto de entrada unificado en el puerto 80. Los datos persistentes se almacenan en un volumen Docker denominado `postgres_data`.

[Insertar diagrama de arquitectura del sistema mostrando los 4 contenedores Docker y sus conexiones]

3.2.2. Backend: FastAPI

El backend se implementó como una aplicación monolítica en Python utilizando el framework FastAPI (versión 0.115.6) con el servidor ASGI Uvicorn (versión 0.34.0). La aplicación se estructura en los siguientes módulos principales, cada uno registrado como un router independiente en el punto de entrada (`app/main.py`):

Router	Prefijo API	Responsabilidad
auth	<code>/api/v1/auth</code>	Autenticación JWT, login, refresh de tokens
frontend_core	<code>/api/v1</code>	CRUD de animales, alertas, tareas, lactaciones, predicciones, calidad, incidencias, empleados, maquinaria, zonas
weather	<code>/api/v1/weather</code>	Consulta y sincronización de datos meteorológicos (AEMET)
simulation	<code>/api/v1/simulation</code>	Generación de datos operativos simulados para demos
audit	<code>/api/v1/audit</code>	Registro de auditoría de operaciones
orders	<code>/api/v1/orders</code>	Gestión de pedidos de insumos

shifts	/api/v1/shifts	Planificación de turnos y asignaciones
handovers	/api/v1/handovers	Relevos entre turnos con resúmenes automáticos
health	/health	Comprobación de estado del servicio

Las dependencias del backend se gestionan mediante un archivo requirements.txt que incluye: SQLAlchemy 2.0.50 como ORM, psycopg 3.x como driver PostgreSQL nativo, Pydantic 2.10+ para validación de esquemas, httpx para peticiones HTTP asíncronas (integración AEMET), Faker para generación de datos de demostración, python-aemet 0.4.1 como wrapper de la API de AEMET, bcrypt 4.0.1 y passlib para el hashing de contraseñas, python-jose para la firma y verificación de tokens JWT, y pytest como framework de testing.

Cabe destacar que el proyecto no incluye dependencias de aprendizaje automático (scikit-learn, XGBoost, TensorFlow u otras). Las predicciones se implementan mediante heurísticas aritméticas basadas en datos de lactación y estado sanitario del animal, como se detalla en la sección 4.6.

3.2.3. Frontend: Next.js + React

La interfaz de usuario se desarrolló con Next.js 16.2.6 y React 19.2.6, utilizando TypeScript 6.0 para el tipado estático. La gestión de estado global se realiza mediante Zustand 5.0.13, que mantiene en un store centralizado el token JWT, los datos del usuario autenticado, el rol activo y la zona seleccionada, con persistencia en localStorage. La comunicación con la API se gestiona mediante TanStack Query 5.100.14, que proporciona caché automática, revalidación en segundo plano y gestión de estados de carga y error.

El diseño visual se basa en TailwindCSS 4.3 como framework de utilidades CSS, complementado con el paquete lucide-react 1.16 para iconografía consistente. No se emplean bibliotecas de componentes prefabricadas (como Material UI o Ant Design);

todos los componentes de interfaz se desarrollaron a medida para ajustarse a los requisitos operativos de una explotación ganadera.

3.2.4. Base de datos: PostgreSQL 15

El sistema de gestión de bases de datos seleccionado es PostgreSQL 15 (imagen Alpine). El esquema relacional se define en el archivo database/init.sql y se complementa con los modelos ORM de SQLAlchemy 2.0 definidos en app/models/tools4milk.py. Las migraciones se aplican mediante un script Python propio (scripts/apply_migrations.py), ejecutado automáticamente en el arranque del contenedor backend.

El esquema comprende más de 25 tablas que cubren los siguientes dominios funcionales:

Dominio	Tablas principales	Descripción
Ganadería	animales, lactaciones, eventos_reproductivos, genomica	Registro individual de animales, ciclos productivos y datos genómicos
Sanidad	eventos_sanitarios, tratamientos_activos, eventos_sanitarios_recria	Historial clínico, tratamientos farmacológicos con checkboxes JSONB y scoring clínico de recría
Operativa	tareas_catalogo, tareas_recurrentes, tareas_ejecuciones	Catálogo de tareas, reglas de recurrencia y ejecuciones individuales
Alertas	alertas_umbrales, alertas	Umbrales configurables por métrica y alertas generadas con trazabilidad de resolución
Incidencias	incidencias	Registro de incidencias con severidad, estado, asignación y acciones (JSONB)
Personal	empleados, turnos, asignaciones_turno, resúmenes_relevo	Gestión de empleados, planificación de turnos y relevos estructurados
Logística	pedidos	Ciclo de vida de pedidos de insumos (solicitud → aprobación)

		→ recepción)
IoT / Telemetría	lecturas_robot_orden, lecturas_carro_mezclador, lecturas_meteorologia	Series temporales preparadas para ingesta de datos de sensores y robots
Infraestructura	zonas, maquinaria, boxes_recria	Topología de la explotación y equipamiento
Auditoría	audit_log	Registro inmutable de operaciones con hash SHA-256

Se utiliza el tipo JSONB de PostgreSQL en varios modelos para almacenar estructuras flexibles: checkboxes de seguimiento de tratamientos, acciones registradas en incidencias, detalles de eventos reproductivos y alertas de boxes de recría. Esta decisión permite evolucionar el esquema de datos sin necesidad de migraciones destructivas.

Es relevante señalar que, aunque el archivo init.sql contiene referencias comentadas a la extensión TimescaleDB y a la creación de hypertables, estas funcionalidades no están activas en la versión actual del sistema. Las tablas de series temporales (lecturas_robot_orden, lecturas_carro_mezclador, lecturas_meteorologia) operan como tablas PostgreSQL estándar con clave primaria compuesta.

3.3. Seguridad y control de acceso

El sistema implementa un esquema de autenticación basado en JSON Web Tokens (JWT) con las siguientes características técnicas:

Las contraseñas se almacenan hasheadas mediante bcrypt a través de la biblioteca passlib. Los tokens de acceso se generan con python-jose utilizando el algoritmo HS256, incluyendo claims estándar (sub, exp, iat) y un identificador único de token (jti) generado con UUID4. El middleware de autenticación (get_current_user) extrae el token del header Authorization (esquema Bearer), lo verifica contra la clave secreta del servidor y recupera el usuario correspondiente de la base de datos. Los usuarios inactivos son rechazados incluso con un token válido.

El control de acceso basado en roles (RBAC) se implementa en dos capas complementarias. En el backend, la función `require_rol` genera dependencias FastAPI que restringen endpoints específicos a conjuntos de roles permitidos. En el frontend, el módulo `role-capabilities.ts` define 35 capacidades granulares (`view_dashboard`, `manage_animals`, `resolve_alert`, `run_simulation`, entre otras) asignadas a cada rol mediante conjuntos explícitos. El sistema define cuatro roles:

Rol	Descripción	Capacidades principales
admin	Administrador del sistema	Acceso completo a todas las funcionalidades (35 capacidades)
veterinario	Personal clínico	Gestión de animales, tratamientos, lactaciones, calidad, alertas, predicciones e incidencias
operario	Personal de campo	Tareas, incidencias, alertas (solo lectura), relevos, maquinaria y vista de animales
alimentacion	Responsable de nutrición	Animales, lactaciones, calidad, tareas, pedidos, incidencias y maquinaria

La pantalla de login incluye cinco usuarios de demostración preconfigurados (`admin`, `roberto.castro`, `operario.zona`, `laura.fernandez`, `dr.mendez`) con contraseña compartida, que se siembran automáticamente en el arranque del sistema mediante la función `seed_demo_user`.

3.4. Integración con datos meteorológicos (AEMET)

El sistema integra datos meteorológicos reales procedentes de la Agencia Estatal de Meteorología (AEMET) a través de su API REST pública (`opendata.aemet.es`). El cliente, implementado en la clase `AemetClient` (`app/services/aemet_client.py`), realiza las siguientes operaciones:

1) Solicita la predicción diaria específica para el municipio configurado (Villalba, Lugo, código 27065) enviando la API key como parámetro de autenticación. 2) Recibe una URL temporal con los datos en formato JSON, los descarga y parsea. 3) Extrae de cada día de predicción: temperatura (media de máxima y mínima), humedad relativa (media de máxima y mínima), precipitación máxima por periodo y velocidad del viento. 4) Realiza un upsert en la tabla `lecturas_meteorologia`, insertando registros nuevos o actualizando los existentes para evitar duplicados.

La integración requiere una API key válida de AEMET configurada como variable de entorno (`AEMET_API_KEY`). En ausencia de esta clave, el sistema omite la sincronización y devuelve un estado "skipped" sin generar error. Las peticiones HTTP se realizan mediante `httpx` con un timeout de 20 segundos.

3.5. Servicio de simulación de datos operativos

Para facilitar la demostración y validación del sistema sin depender de datos reales de producción, se implementó un servicio de simulación (`app/services/simulation_service.py`) que genera datos operativos ficticios mediante la biblioteca `Faker`. Este servicio crea registros de tareas, alertas, incidencias, actualizaciones de lactación y estados de maquinaria, respetando las restricciones del esquema relacional y las reglas de negocio definidas en los modelos.

La simulación se activa bajo demanda a través del endpoint `/api/v1/simulation` y está controlada por el parámetro de configuración `enable_simulation`. Su uso está restringido a usuarios con permisos de administración (`capacidad_run_simulation`). Este mecanismo permite poblar el sistema con datos coherentes para pruebas funcionales, demostraciones a stakeholders y validación de flujos de interfaz de usuario sin comprometer la integridad de datos reales.

3.6. Herramientas de desarrollo y testing

El desarrollo se realizó con las siguientes herramientas y prácticas: control de versiones con `Git`, contenedorización completa con `Docker` y `Docker Compose`, linting frontend con `ESLint` y verificación de tipos con `TypeScript` (`tsc --noEmit`). El backend incluye `pytest` como framework de testing con soporte asíncrono (`pytest-asyncio`). La validación

de configuración en producción se realiza automáticamente al arrancar la aplicación, verificando que la clave secreta tenga una longitud mínima de 32 caracteres y que los orígenes CORS estén explícitamente configurados.

4. Resultados

En esta sección se describe el producto software obtenido como resultado del trabajo de desarrollo. Se presenta cada módulo funcional de la plataforma Tools4Milk tal como se encuentra implementado en el código fuente, incluyendo su interfaz de usuario, su lógica de negocio y sus endpoints API. Todos los elementos descritos son verificables en el repositorio del proyecto.

4.1. Vista general: Dashboard operativo

El dashboard constituye la pantalla principal del sistema tras el inicio de sesión. Presenta un resumen ejecutivo del estado de la explotación, consumido desde el endpoint GET /api/v1/dashboard. Este endpoint agrega en tiempo real el número de alertas activas, el total de animales registrados, las tareas pendientes y las incidencias abiertas. La vista se adapta al rol del usuario: todos los roles tienen acceso de lectura (capacidad view_dashboard), pero la información mostrada se filtra según las capacidades del perfil autenticado.

[Insertar captura de pantalla del dashboard general]

4.2. Gestión de animales

El módulo de ganadería permite el registro y seguimiento individual de cada animal de la explotación. La interfaz presenta un listado filtrable con búsqueda por crotal oficial, que constituye el identificador único de cada animal. La ficha individual de un animal muestra: datos de identificación (crotal, nombre, raza, sexo, fecha de nacimiento), estado productivo (recría, producción, secado, baja), estado reproductivo, fecha de entrada en la explotación y notas adicionales.

El backend expone endpoints CRUD completos: listado paginado (GET /api/v1/animals), creación (POST /api/v1/animals), detalle (GET /api/v1/animals/{id}), actualización (PATCH /api/v1/animals/{id}) y búsqueda por crotal (GET /api/v1/animals/search-by-crotal/{crotal}). El acceso está controlado por las capacidades view_animals y manage_animals, lo que permite que un operario visualice la ficha de un animal pero no la modifique.

El modelo de datos asocia a cada animal sus lactaciones, tratamientos activos, eventos sanitarios, eventos reproductivos y datos genómicos, proporcionando una visión longitudinal del historial productivo y clínico del animal. La relación madre-hijo se modela mediante una clave foránea autoreferencial en la tabla animales.

[Insertar captura de pantalla del listado de animales]

[Insertar captura de pantalla de la ficha individual de un animal]

4.3. Sistema de alertas configurables

El módulo de alertas opera sobre un sistema de umbrales configurables almacenados en la tabla `alertas_umbrales`. Cada umbral define: un código identificativo, la métrica monitoreada, el operador de comparación, el valor límite, la unidad, el nivel de alerta y los canales de notificación (pantalla TV, tablet, campo para WhatsApp —actualmente no implementado—). Las alertas generadas se almacenan en la tabla `alertas` con trazabilidad completa: timestamp de generación, animal y zona afectados, estado de resolución y empleado que la resuelve.

La API expone endpoints para listar alertas (con filtro por activas), crear alertas manuales, consultar alertas de un animal específico, obtener alertas críticas y generar alertas automáticas mediante el endpoint `POST /api/v1/alerts/generate`. La resolución de alertas (review) se restringe a usuarios con la capacidad `resolve_alert` (roles admin y veterinario).

La interfaz frontend presenta las alertas en una vista con indicadores visuales de severidad (colores diferenciados para niveles info, aviso, alerta y crítica) y permite la revisión y resolución directa desde la pantalla.

[Insertar captura de pantalla del panel de alertas]

4.4. Gestión de tareas operativas y Lean Farming

El sistema implementa un modelo de gestión de tareas inspirado en principios Lean, estructurado en tres niveles: catálogo de tareas (`tareas_catalogo`), reglas de recurrencia (`tareas_recurrentes`) y ejecuciones individuales (`tareas_ejecuciones`). El catálogo define tareas tipo con su código, nombre, descripción, cualificación requerida y duración estimada en minutos. Las reglas de recurrencia asocian una tarea del catálogo a una zona

o maquinaria específica con una expresión de frecuencia. Las ejecuciones registran cada instancia concreta con estados (pendiente, en_curso, completada) y marcas temporales de planificación, inicio y fin.

La interfaz de Lean Farming (página leanfarming, 505 líneas de código) proporciona una vista tipo Kanban o tablero de gestión visual de las tareas del día, organizadas por zona y estado. La página de tareas (tasks, 439 líneas) complementa con un listado completo y filtrable de todas las ejecuciones. Ambas vistas están accesibles para operarios (manage_tasks, create_task, complete_task) y administradores.

[Insertar captura de pantalla de la vista Lean Farming]

[Insertar captura de pantalla del listado de tareas]

4.5. Control de calidad de leche

El módulo de calidad de leche presenta un resumen estadístico del estado productivo del rebaño, calculado a partir de los registros de lactaciones activas. El endpoint GET /api/v1/quality invoca el servicio lactations_service.quality_summary, que agrega métricas de las lactaciones en curso: producción total media, distribución por número de lactación y estadísticas descriptivas del rebaño.

La interfaz frontend (página quality, 567 líneas de código) presenta estos datos mediante indicadores KPI, tablas resumen y visualizaciones que permiten al responsable de nutrición y al veterinario evaluar rápidamente el rendimiento productivo global. El acceso se controla mediante la capacidad view_quality, disponible para los roles admin, veterinario y alimentacion.

[Insertar captura de pantalla de la pantalla de calidad de leche]

4.6. Módulo de predicciones

El módulo de predicciones proporciona estimaciones individuales de producción y riesgo sanitario para cada animal. Es importante destacar que, en la versión actual del MVP, estas predicciones se calculan mediante heurísticas aritméticas deterministas, no mediante modelos de aprendizaje automático. El endpoint GET /api/v1/predictions/{animal_id} implementa la siguiente lógica:

1) Producción base: se calcula como la producción total de la lactación activa dividida entre 305 días (duración estándar de lactación). 2) Penalización por tratamiento: se aplica un factor de reducción del 8 % si el animal tiene tratamientos activos. 3) Penalización por alertas: se descuenta un 3 % por cada alerta pendiente, con un tope máximo del 12 %. 4) Producción esperada: $\text{producción_base} \times (1 - \text{penalización_tratamiento} - \text{penalización_alertas})$. 5) Rango de predicción: $\pm 7\%$ sobre el valor esperado. 6) Serie diaria: 7 valores generados aplicando factores fijos [0.98, 0.99, 1.0, 1.01, 1.0, 1.02, 1.01] sobre la producción esperada. 7) Nivel de riesgo: alto si hay tratamientos activos, medio si hay alertas pendientes, bajo en caso contrario.

La respuesta incluye también datos de composición de leche (grasa, proteína, lactosa) y riesgos sanitarios específicos (mastitis), aunque estos valores son actualmente fijos (placeholder) y no se calculan a partir de datos reales. Los índices de confianza reportados (0.82 con lactación, 0.45 sin ella) son valores estáticos incorporados en el código.

La interfaz de predicciones (página predictions, 387 líneas de código) permite seleccionar un animal y visualizar sus estimaciones de producción, tendencia, nivel de riesgo y factores de riesgo identificados. Esta vista está restringida a los roles admin y veterinario (capacidad view_predictions).

[Insertar captura de pantalla de la pantalla de predicciones para un animal]

4.7. Gestión de incidencias

El módulo de incidencias permite registrar, clasificar y seguir eventos no planificados que afectan a la operativa de la explotación. Cada incidencia se caracteriza por: tipo y subtipo, severidad (baja, media, alta, crítica), estado (abierta, en_progreso, resuelta, cerrada), título descriptivo, zona y/o maquinaria afectada, animal relacionado (si aplica), empleado que reporta y empleado asignado, timestamps de apertura y cierre, y un campo JSONB de acciones que registra la secuencia de intervenciones realizadas.

La interfaz de incidencias (página incidents, 632 líneas de código, la más extensa del frontend) ofrece funcionalidades de creación, filtrado por estado y severidad, detalle y

resolución. El acceso se controla mediante las capacidades `manage_incidents` y `create_incident`, disponibles para todos los roles del sistema.

[Insertar captura de pantalla del listado de incidencias]

4.8. Planificación de turnos y relevos

El sistema de turnos permite definir periodos de trabajo (mañana, tarde, noche) con hora de inicio y fin, y asignar empleados a cada turno con zona y rol específicos. La tabla `turnos` almacena la definición del turno (con restricción de unicidad por fecha y tipo), mientras que `asignaciones_turno` vincula empleados individuales al turno correspondiente.

El módulo de relevos (`handovers`) genera resúmenes estructurados de cambio de turno que incluyen: incidencias abiertas, tareas pendientes y alertas pendientes en el momento del relevo, notas del turno saliente, y confirmación del turno entrante con timestamp y empleado que confirma. Esta funcionalidad, implementada en el modelo `ResumenRelevo` con campos `JSONB`, garantiza la continuidad operativa entre turnos.

La interfaz de turnos (página `shifts`, 660 líneas de código) permite la creación y visualización de turnos con sus asignaciones. La página de relevos (`handover`, 273 líneas) presenta el resumen del relevo con opción de confirmación. Adicionalmente, existe una vista de relevos optimizada para tablet (`handover/tablet`) que simplifica la interacción en dispositivos de campo.

[Insertar captura de pantalla de la planificación de turnos]

[Insertar captura de pantalla del resumen de relevo]

4.9. Gestión de pedidos de insumos

El módulo de pedidos gestiona el ciclo de vida completo de las solicitudes de insumos: desde la creación de la solicitud hasta la recepción del material. Cada pedido registra el insumo solicitado, cantidad y unidad, estado (solicitado, aprobado, recibido, cancelado), empleado solicitante, proveedor, coste estimado y coste real, y timestamps de cada transición de estado.

La interfaz (página orders, 587 líneas de código) permite crear solicitudes, aprobar pedidos pendientes y registrar la recepción de materiales. El acceso se controla mediante las capacidades manage_orders y create_order, asignadas a los roles admin y alimentacion.

[Insertar captura de pantalla de la gestión de pedidos]

4.10. Modo TV para pantallas de zona

El sistema incluye un modo TV diseñado para pantallas de visualización ubicadas en las zonas de trabajo de la explotación. Este modo presenta información operativa relevante (alertas activas, tareas del turno, estado de maquinaria) en formato de solo lectura y con diseño optimizado para pantallas grandes. La configuración de qué zonas disponen de pantalla TV se gestiona mediante el campo tiene_pantalla_tv del modelo Zona.

Existen dos vistas TV implementadas: una vista general (tv) y una vista específica de turnos (tv/shifts). El acceso al modo TV se controla mediante la capacidad view_tv_global, disponible para todos los roles del sistema.

[Insertar captura de pantalla del modo TV]

4.11. Módulos de administración y configuración

El sistema incluye varios módulos administrativos accesibles según el rol del usuario:

Gestión de usuarios y empleados (página management, 961 líneas de código): la pantalla más extensa en términos de código frontend. Permite la administración de empleados (alta, edición, cualificaciones, estado activo/baja) y, para administradores, la gestión de cuentas de usuario del sistema. Diferencia entre empleados (personal operativo de la explotación) y usuarios (cuentas de acceso al sistema).

Gestión de zonas (página zones, 327 líneas): permite crear y editar zonas de la explotación, configurando si disponen de pantalla TV o tablet.

Registro de auditoría (página audit-log, 337 líneas): presenta un listado cronológico de todas las operaciones registradas en el audit_log, incluyendo tabla afectada, tipo de operación (INSERT, UPDATE, DELETE), datos anteriores y nuevos (JSONB) y hash SHA-256 para garantizar la integridad del registro.

Integración (página integration, 286 líneas): panel de estado de las integraciones externas del sistema, incluyendo la conexión con AEMET y el estado del servicio de simulación.

Configuración (página settings, 406 líneas): parámetros generales del sistema. Perfil de usuario (página profile, 396 líneas): visualización y edición del perfil del usuario autenticado.

Informe operativo (página report, 452 líneas): generación de informes agregados del estado de la explotación, consolidando datos de producción, alertas, tareas e incidencias.

[Insertar captura de pantalla de la gestión de empleados]

[Insertar captura de pantalla del registro de auditoría]

4.12. Datos meteorológicos en la interfaz

Los datos meteorológicos sincronizados desde AEMET se exponen en la interfaz a través de los endpoints GET /api/v1/weather/current y GET /api/v1/weather/forecast, que consultan las últimas lecturas almacenadas en la tabla lecturas_meteorologia. La información meteorológica se integra como contexto operativo, permitiendo al personal de la explotación correlacionar visualmente condiciones ambientales (temperatura, humedad, precipitación, viento) con eventos productivos o sanitarios.

[Insertar captura de pantalla del widget meteorológico]

5. Limitaciones del MVP y trabajo futuro

5.1. Limitaciones de la versión actual

El sistema presentado constituye un MVP funcional cuyo alcance se acota deliberadamente para validar la viabilidad técnica de la plataforma. Se identifican las siguientes limitaciones en la implementación actual:

Ausencia de modelos de aprendizaje automático. Las predicciones de producción y riesgo sanitario se basan en heurísticas aritméticas deterministas. No se han integrado algoritmos de regresión, clasificación ni series temporales. El esquema de datos está preparado para alimentar modelos futuros (tablas de series temporales, datos genómicos, historial sanitario longitudinal), pero el entrenamiento y despliegue de modelos queda fuera del alcance de esta versión.

Sin ingesta de datos IoT en tiempo real. Las tablas de lecturas de robot de ordeño y carro mezclador están definidas en el esquema pero no reciben datos automáticamente. No existe un pipeline de ingesta (MQTT, Kafka u otro) que conecte los equipos de la explotación con la base de datos. Los datos de demostración se generan mediante el servicio de simulación.

TimescaleDB no activado. Aunque el esquema incluye referencias comentadas a TimescaleDB y hypertables, la extensión no está habilitada. Las series temporales operan como tablas PostgreSQL estándar, lo que puede limitar el rendimiento de consultas agregadas sobre volúmenes elevados de datos.

Sin despliegue en la nube. El sistema se ejecuta localmente mediante Docker Compose. No se ha implementado infraestructura en Azure, AWS ni otro proveedor cloud. No existe pipeline CI/CD, monitorización de producción ni escalado automático.

Notificaciones push no implementadas. El modelo de alertas incluye un campo `push_whatsapp` que no está conectado a ningún servicio de mensajería. Las alertas se muestran exclusivamente en la interfaz web.

Sin migración automatizada con Alembic. Las migraciones de esquema se gestionan mediante un script Python propio (`apply_migrations.py`) que ejecuta sentencias ALTER

TABLE directas. No se utiliza Alembic ni otro sistema de control de versiones de esquema.

Composición de leche estática en predicciones. Los valores de grasa, proteína y lactosa en la respuesta de predicciones son placeholders fijos (0), no calculados a partir de datos analíticos reales.

5.2. Líneas de trabajo futuro

A partir del MVP desarrollado, se identifican las siguientes líneas de evolución prioritarias:

Integración de modelos predictivos. Incorporar modelos de aprendizaje automático entrenados con datos reales de producción, calidad y sanidad. Se propone explorar modelos de regresión para predicción de curvas de lactación, clasificadores para detección temprana de mastitis basados en conductividad y recuento de células somáticas, y modelos de series temporales para optimización de raciones alimentarias. Los frameworks candidatos incluyen scikit-learn para modelos tabulares y XGBoost para gradient boosting.

Pipeline de ingesta IoT. Implementar un broker MQTT (Mosquitto o similar) para recibir datos en tiempo real de robots de ordeño, carros mezcladores y sensores ambientales. Diseñar un pipeline ETL que valide, transforme y persista las lecturas en las tablas de series temporales existentes.

Activación de TimescaleDB. Habilitar la extensión TimescaleDB y convertir las tablas de series temporales en hypertables, habilitando compresión automática, políticas de retención de datos y funciones de agregación continua para consultas analíticas de alto rendimiento.

Despliegue en la nube. Migrar la infraestructura a Azure Container Instances o Azure Kubernetes Service, implementar un pipeline CI/CD con GitHub Actions, configurar monitorización con Application Insights y establecer políticas de backup automatizado.

Notificaciones y comunicación. Integrar un servicio de notificaciones push (WhatsApp Business API o Telegram) para alertas críticas, permitiendo que el personal reciba avisos en tiempo real sin necesidad de consultar la interfaz web.

Simulador «leche a la carta». Desarrollar un módulo interactivo que permita simular el impacto de cambios en la ración alimentaria sobre la composición de la leche (grasa, proteína, lactosa), apoyado en modelos nutricionales calibrados con datos de la explotación.

Anexo: Auditoría de coherencia técnica

Se realizó una auditoría exhaustiva comparando la descripción contenida en el TFM original con el estado real del código fuente del proyecto. A continuación se documentan las discrepancias identificadas y las correcciones aplicadas en la versión revisada del documento.

Elemento del TFM original	Estado real en el código	Acción correctiva
Arquitectura de microservicios (4 servicios independientes)	Monolito modular FastAPI con un único proceso	Reescrito como arquitectura monolítica modular
TimescaleDB como motor de series temporales	PostgreSQL 15 estándar; TimescaleDB comentado en init.sql	Corregido a PostgreSQL 15; TimescaleDB citado como trabajo futuro
Despliegue en Azure (Container Instances, AKS)	Docker Compose local, sin ningún componente Azure	Eliminadas referencias a Azure; descrito despliegue Docker local
Modelos ML: XGBoost, Random Forest, LSTM	Heurísticas aritméticas en frontend_core.py (líneas 425-434)	Reescrito como predicciones heurísticas; ML movido a trabajo futuro
ETL pipeline con Apache Airflow / Prefect	No existe pipeline ETL; datos de simulación con Faker	Eliminado; descrito servicio de simulación real
MQTT broker para IoT	No existe broker MQTT ni ingesta IoT	Eliminado; mencionado como trabajo futuro
Redis como caché	No hay Redis en docker-compose ni en requirements	Eliminado del documento
Alembic para migraciones	Script propio apply_migrations.py	Corregido a script propio de migraciones
Notificaciones WhatsApp	Campo push_whatsapp existe pero sin integración real	Documentado como campo preparatorio sin implementación
SHAP values para explicabilidad	No existe código de explicabilidad	Eliminado del documento
Simulador «leche a la carta»	No implementado	Movido a trabajo futuro
SCD Type 2 para dimensiones	No implementado; tablas	Eliminado del documento

	estándar	
CQRS (Command Query Responsibility Segregation)	No implementado; endpoints CRUD estándar	Eliminado del documento
Conectores DeLaval / Lely	No implementados	Eliminado; mencionado como trabajo futuro
Scheduler automático de tareas	Tareas recurrentes definidas pero sin scheduler automático	Corregido: modelo de recurrencia sin ejecución automática

Todas las correcciones aplicadas siguen el principio de describir exclusivamente funcionalidades verificables en el código fuente. Los elementos no implementados se reclasifican como limitaciones del MVP actual o como líneas de trabajo futuro, sin presentarlos como resultados obtenidos.